

Predicción de volatilidad financiera

Apuntes teóricos y computacionales para la Semana 3

Descarga y exploración inicial de datos financieros en Python

Objetivo de la semana. Descargar formalmente la serie histórica de SPY mediante Python, comprender la estructura de la base obtenida y realizar una revisión inicial de sus columnas, fechas, valores faltantes, duplicados y frecuencia temporal. Al terminar esta semana, la estudiante deberá poder describir la cobertura y la calidad inicial de la base, además de conservar una copia reproducible de los datos originales.

Índice

1. Contexto de la Semana 3	3
2. Objetivos de aprendizaje	3
3. Del activo seleccionado a la base de datos	4
4. Preparación del entorno computacional	5
5. Parámetros de la descarga	5
6. Descarga histórica con <code>yfinance</code>	6
7. Estructura de un <code>DataFrame</code> financiero	7
8. Interpretación de las columnas	8

9. Índice temporal y cobertura de la serie	9
10.Días naturales y días de operación	9
11.Identificación de valores faltantes	11
12.Identificación de fechas duplicadas	11
13.Verificaciones básicas de consistencia	12
14.Visualización preliminar	13
15.Resumen automático de la descarga	14
16.Conservación de los datos originales	15
17.Ficha técnica de la descarga	16
18.Producto esperado al final de la semana	17
19.Preguntas de repaso	17
20.Conclusión	18

1. Contexto de la Semana 3

En la Semana 1 se revisaron los conceptos básicos de precios, rendimientos y volatilidad. En la Semana 2 se compararon distintas fuentes de datos y se fijaron las decisiones principales del proyecto:

Elemento	Decisión
Fuente principal	Yahoo Finance mediante <code>yfinance</code> .
Activo principal	SPY.
Tipo de activo	ETF que sigue al índice S&P 500.
Periodo	2015-01-01 a 2025-12-31.
Frecuencia	Diaria.
Variable principal	Precio de cierre ajustado.

La Semana 3 transforma estas decisiones en una base de datos concreta. El propósito no es todavía calcular rendimientos ni volatilidad, sino responder preguntas más elementales:

1. ¿La descarga se realizó correctamente?
2. ¿Qué columnas contiene la base?
3. ¿Qué representa cada fila?
4. ¿Cuál es la primera y la última fecha disponibles?
5. ¿Existen valores faltantes?
6. ¿Existen fechas duplicadas?
7. ¿Las fechas están ordenadas?
8. ¿La frecuencia observada es compatible con un activo bursátil?
9. ¿Existen inconsistencias evidentes entre los precios diarios?

Estas preguntas forman parte de una **auditoría inicial de datos**. Antes de aplicar cualquier transformación estadística, se debe conocer la estructura y las limitaciones de la información disponible.

Delimitación de la semana. Durante la Semana 3 se identificarán posibles problemas, pero no se corregirán todavía. El ordenamiento definitivo, la eliminación de duplicados y el tratamiento de valores faltantes corresponden a la Semana 4.

2. Objetivos de aprendizaje

Al finalizar la semana, la estudiante deberá ser capaz de:

1. preparar un entorno básico de trabajo en Python;
2. definir de forma explícita los parámetros de una descarga;
3. descargar datos históricos con `yfinance`;
4. verificar si la descarga produjo una base no vacía;
5. reconocer la estructura de un objeto `DataFrame`;
6. interpretar las columnas `Open`, `High`, `Low`, `Close`, `Adj Close` y `Volume`;
7. examinar el índice temporal de la base;
8. identificar valores faltantes y fechas duplicadas;
9. distinguir entre días naturales y días de operación;
10. realizar verificaciones elementales de consistencia;
11. construir visualizaciones preliminares de precio y volumen;
12. guardar y recuperar una copia de los datos originales.

3. Del activo seleccionado a la base de datos

En la Semana 2 se eligió SPY como activo principal. Esta elección define el símbolo que deberá utilizarse en la descarga. También se estableció el periodo de análisis entre el 1 de enero de 2015 y el 31 de diciembre de 2025.

Una base financiera diaria puede entenderse como una tabla

$$\mathcal{D} = \{(t_i, X_i) : i = 1, \dots, n\},$$

donde t_i representa una fecha de operación y X_i contiene las variables observadas ese día. Para la base descargada, cada vector X_i tendrá la forma

$$X_i = (O_i, H_i, L_i, C_i, A_i, V_i),$$

donde:

- O_i es el precio de apertura;
- H_i es el precio máximo;
- L_i es el precio mínimo;
- C_i es el precio de cierre;
- A_i es el precio de cierre ajustado;
- V_i es el volumen de operación.

En esta semana se estudiará la tabla como una estructura de datos. Las transformaciones financieras se introducirán más adelante.

4. Preparación del entorno computacional

Se trabajará en un cuaderno de Jupyter. El notebook puede llamarse

03_descarga_exploracion_inicial.ipynb.

Las bibliotecas necesarias son:

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import yfinance as yf
4 from pathlib import Path
```

Cada biblioteca tendrá una función concreta:

Biblioteca	Uso en esta semana
pandas	Manipulación e inspección de tablas de datos.
matplotlib	Construcción de gráficas preliminares.
yfinance	Descarga de datos históricos de mercado.
pathlib	Creación de carpetas y manejo reproducible de rutas.

Registro de versiones

Las bibliotecas pueden cambiar con el tiempo. Por ello, conviene registrar al menos las versiones de `pandas` y `yfinance` utilizadas en el análisis.

```
1 print("Version de pandas:", pd.__version__)
2 print("Version de yfinance:", yf.__version__)
```

Este registro ayuda a reproducir el notebook si en el futuro cambia el comportamiento de alguna función.

5. Parámetros de la descarga

Los parámetros principales deben definirse en una celda independiente:

```
1 ticker = "SPY"
2 fecha_inicio = "2015-01-01"
3 fecha_fin = "2026-01-01"
```

La fecha final se escribe como 2026-01-01 porque el argumento `end` de `yf.download` se interpreta como un límite exclusivo. De esta forma, la solicitud cubre las observaciones disponibles hasta el 31 de diciembre de 2025.

Separar los parámetros del resto del código tiene dos ventajas:

1. permite modificar el activo o el periodo sin reescribir todo el notebook;
2. hace explícitas las decisiones metodológicas del proyecto.

Buena práctica. Los símbolos, fechas, ventanas y rutas importantes no deben quedar ocultos dentro de instrucciones largas. Conviene definirlos como variables al inicio del notebook.

6. Descarga histórica con yfinance

La descarga se realizará mediante la función `yf.download`:

```

1 datos = yf.download(
2     ticker,
3     start=fecha_inicio,
4     end=fecha_fin,
5     interval="1d",
6     auto_adjust=False,
7     multi_level_index=False,
8     progress=False
9 )

```

Los argumentos utilizados tienen la siguiente interpretación:

Argumento	Interpretación
<code>ticker</code>	Símbolo financiero del activo.
<code>start</code>	Fecha inicial solicitada.
<code>end</code>	Límite final exclusivo de la descarga.
<code>interval="1d"</code>	Una observación por día de mercado.
<code>auto_adjust=False</code>	Conserva separadas las columnas de cierre y cierre ajustado.
<code>multi_level_index=False</code>	Solicita columnas simples para un solo activo.
<code>progress=False</code>	Oculta la barra de progreso de la descarga.

El resultado se almacena en el objeto `datos`. En condiciones normales, este objeto será un `DataFrame` de `pandas`.

Verificación de una descarga no vacía

Antes de continuar, se debe comprobar que la base contiene observaciones:

```

1 if datos.empty:
2     raise ValueError(
3         "No se descargaron datos. Revise el ticker, "
4         "las fechas y la conexión a internet."

```

```

5 )
6
7 print("La descarga se realizo correctamente.")

```

Una tabla vacía puede deberse a un símbolo incorrecto, una conexión fallida, un intervalo no disponible o un periodo mal especificado. Continuar el análisis con una tabla vacía produciría errores posteriores difíciles de interpretar.

7. Estructura de un DataFrame financiero

Un **DataFrame** es una tabla con filas, columnas e índice. En la base descargada:

- cada fila representa una fecha de operación;
- cada columna representa una variable financiera;
- el índice contiene las fechas;
- los valores contienen precios o volumen.

La exploración inicial puede comenzar con las siguientes instrucciones:

```

1 type(datos)
2 datos.shape
3 datos.head()
4 datos.tail()
5 datos.columns
6 datos.index
7 datos.info()

```

Cada instrucción responde una pregunta distinta.

Instrucción	Pregunta que responde
<code>type(datos)</code>	¿Qué tipo de objeto se descargó?
<code>datos.shape</code>	¿Cuántas filas y columnas tiene la base?
<code>datos.head()</code>	¿Cómo son las primeras observaciones?
<code>datos.tail()</code>	¿Cómo son las últimas observaciones?
<code>datos.columns</code>	¿Qué variables están disponibles?
<code>datos.index</code>	¿Cómo están almacenadas las fechas?
<code>datos.info()</code>	¿Qué tipos de datos tienen las columnas y cuántos valores no nulos contienen?

Primeras observaciones

```

1 datos.head()

```

Por omisión, `head()` muestra las primeras cinco filas. Si se desea otro número, puede utilizarse, por ejemplo,

```
1 datos.head(10)
```

Últimas observaciones

```
1 datos.tail()
```

Esta instrucción ayuda a verificar si la cobertura final coincide con lo esperado.

Dimensiones

```
1 print("Numero de filas:", datos.shape[0])
2 print("Numero de columnas:", datos.shape[1])
```

Si la base tiene n filas y p columnas, entonces `datos.shape` devuelve el par

$$(n, p).$$

8. Interpretación de las columnas

La descarga de SPY debe contener, en principio, las columnas siguientes:

Columna	Interpretación
Open	Precio al inicio de la sesión.
High	Precio máximo observado durante la sesión.
Low	Precio mínimo observado durante la sesión.
Close	Precio al final de la sesión.
Adj Close	Precio de cierre ajustado por eventos como dividendos y divisiones.
Volume	Número de unidades negociadas durante la sesión.

Los nombres pueden revisarse con:

```
1 print(datos.columns)
```

Para cada fecha t , los precios diarios deberían satisfacer las relaciones básicas

$$L_t \leq O_t \leq H_t, \quad L_t \leq C_t \leq H_t,$$

y, en particular,

$$L_t \leq H_t.$$

Estas desigualdades se usarán más adelante para detectar inconsistencias evidentes.

Observación. La columna `Adj Close` no tiene que permanecer entre `Low` y `High`, porque incorpora ajustes acumulados que no representan necesariamente el precio negociado durante esa sesión.

9. Índice temporal y cobertura de la serie

En la tabla descargada, la fecha se almacena como índice. Esto puede verificarse con:

```
1 print("Tipo de índice:", type(datos.index))
2 print("Primera fecha:", datos.index.min())
3 print("Última fecha:", datos.index.max())
4 print("Número de observaciones:", len(datos))
```

Lo esperado es que el índice sea de tipo `DatetimeIndex`. Este tipo permite realizar operaciones temporales como comparar fechas, calcular diferencias o identificar días de la semana.

Debe distinguirse entre:

- la fecha inicial solicitada;
- la primera fecha realmente disponible;
- la fecha final exclusiva solicitada;
- la última fecha realmente disponible.

Por ejemplo, si el 1 de enero no hubo operación bursátil, la primera observación del año puede corresponder a un día posterior. Esto no implica necesariamente que exista un error.

Verificación del orden temporal

```
1 print(
2     "Las fechas están ordenadas:",
3     datos.index.is_monotonic_increasing
4 )
```

El valor `True` indica que las fechas aparecen de la más antigua a la más reciente. En esta semana solo se registra el resultado. Si fuera `False`, el ordenamiento definitivo se realizará durante la limpieza de la Semana 4.

10. Días naturales y días de operación

Una frecuencia diaria de mercado no significa que exista una observación para cada día natural. En el caso de `SPY`, normalmente no hay operaciones regulares durante:

- sábados;
- domingos;
- días festivos del mercado;
- cierres extraordinarios.

Por esta razón, una separación de tres días entre dos fechas consecutivas puede corresponder simplemente al fin de semana.

Las diferencias entre fechas consecutivas se calculan con:

```
1 diferencias_fechas = datos.index.to_series().diff()
2
3 frecuencia_observada = (
4     diferencias_fechas
5     .value_counts()
6     .sort_index()
7 )
8
9 frecuencia_observada
```

Para expresar las diferencias en días enteros:

```
1 diferencias_en_dias = diferencias_fechas.dt.days
2
3 diferencias_en_dias.value_counts().sort_index()
```

Esta tabla permite contar cuántas separaciones son de uno, dos, tres o más días.

Día de la semana

El índice temporal también permite identificar el día de la semana:

```
1 dias_semana = pd.Series(
2     datos.index.day_name(),
3     index=datos.index
4 )
5
6 dias_semana.value_counts()
```

Para revisar si existen observaciones en sábado o domingo:

```
1 fin_de_semana = datos.index.dayofweek >= 5
2
3 print(
4     "Observaciones en sabado o domingo:",
5     fin_de_semana.sum()
6 )
```

Para un ETF bursátil como SPY, normalmente se espera que este conteo sea cero.

Punto importante. Una fecha que no aparece en la base no debe clasificarse automáticamente como valor faltante. Primero se debe determinar si era un día de operación.

11. Identificación de valores faltantes

Un valor faltante se representa comúnmente mediante NaN. Para contar los valores faltantes por columna se utiliza:

```
1 datos.isna().sum()
```

También puede calcularse el porcentaje correspondiente:

```
1 revision_faltantes = pd.DataFrame({
2     "Valores faltantes": datos.isna().sum(),
3     "Porcentaje": 100 * datos.isna().mean()
4 })
5
6 revision_faltantes
```

Si una columna contiene m_j valores faltantes de un total de n observaciones, el porcentaje faltante es

$$100 \frac{m_j}{n}.$$

La revisión debe responder:

1. ¿qué columnas tienen faltantes?;
2. ¿cuántos faltantes aparecen en cada columna?;
3. ¿qué porcentaje representan?;
4. ¿los faltantes ocurren en las mismas fechas para varias columnas?;
5. ¿la cantidad parece pequeña o requiere una revisión detallada?

En esta etapa no se aplicará `dropna()` ni `fillna()`. Primero se debe documentar la situación y después decidir un tratamiento apropiado.

12. Identificación de fechas duplicadas

Para una base diaria de un solo activo, se espera una observación por fecha. El número de fechas duplicadas se obtiene mediante:

```
1 numero_duplicadas = datos.index.duplicated().sum()
2
3 print("Numero de fechas duplicadas:", numero_duplicadas)
```

Si el conteo es positivo, pueden mostrarse las filas involucradas:

```
1 if numero_duplicadas > 0:  
2     display(datos[datos.index.duplicated(keep=False)])
```

La opción `keep=False` marca todas las apariciones de una fecha repetida. Durante esta semana las filas solo se identifican. La decisión de conservar, combinar o eliminar observaciones se tomará en la Semana 4.

13. Verificaciones básicas de consistencia

Además de revisar fechas y faltantes, conviene buscar observaciones que contradigan propiedades elementales de los datos diarios.

Precios no positivos

En condiciones normales, los precios deben ser estrictamente positivos:

```
1 columnas_precios = [  
2     "Open",  
3     "High",  
4     "Low",  
5     "Close",  
6     "Adj Close"  
7 ]  
8  
9 (datos[columnas_precios] <= 0).sum()
```

Un conteo positivo requeriría revisar las fechas correspondientes antes de calcular rendimientos logarítmicos, porque el logaritmo no está definido para valores no positivos.

Máximo menor que mínimo

```
1 print(  
2     "Dias con High menor que Low:",  
3     (datos["High"] < datos["Low"]).sum()  
4 )
```

El máximo diario no puede ser menor que el mínimo diario.

Apertura fuera del intervalo diario

```
1 apertura_inconsistente = (  
2     (datos["Open"] < datos["Low"]) |  
3     (datos["Open"] > datos["High"])  
4 )
```

```
5
6 print(
7     "Aperturas fuera del intervalo Low-High:",
8     apertura_inconsistente.sum()
9 )
```

Cierre fuera del intervalo diario

```
1 cierre_inconsistente = (
2     (datos["Close"] < datos["Low"]) |
3     (datos["Close"] > datos["High"])
4 )
5
6 print(
7     "Cierres fuera del intervalo Low-High:",
8     cierre_inconsistente.sum()
9 )
```

Volumen negativo

```
1 print(
2     "Observaciones con volumen negativo:",
3     (datos["Volume"] < 0).sum()
4 )
```

El volumen puede ser cero en algunas bases o instrumentos, pero no debería ser negativo.

Interpretación correcta. Estas verificaciones son filtros elementales, no una garantía absoluta de calidad. Una base puede superar todas las pruebas anteriores y aun contener errores más sutiles.

14. Visualización preliminar

Las gráficas ayudan a detectar tendencias, saltos o periodos con comportamiento inusual. En esta semana se construirán únicamente dos visualizaciones: precio de cierre ajustado y volumen.

Precio de cierre ajustado

```
1 ax = datos["Adj Close"].plot(
2     figsize=(12, 5),
3     title="Precio de cierre ajustado de SPY"
4 )
5
6 ax.set_xlabel("Fecha")
```

```
7 ax.set_ylabel("Precio ajustado")
8 ax.grid(alpha=0.3)
9
10 plt.show()
```

La gráfica permite observar:

- la evolución general del precio;
- periodos de crecimiento o caída;
- cambios bruscos;
- posibles observaciones aisladas;
- la cobertura temporal de la serie.

No debe interpretarse todavía esta gráfica como una medida de volatilidad. La volatilidad se definirá a partir de los rendimientos en semanas posteriores.

Volumen diario

```
1 ax = datos["Volume"].plot(
2     figsize=(12, 4),
3     title="Volumen diario de SPY"
4 )
5
6 ax.set_xlabel("Fecha")
7 ax.set_ylabel("Volumen")
8 ax.grid(alpha=0.3)
9
10 plt.show()
```

Esta gráfica puede mostrar días de actividad inusualmente alta. Sin embargo, una observación extrema no debe eliminarse únicamente por verse grande: puede corresponder a un episodio real del mercado.

15. Resumen automático de la descarga

Para concentrar los principales resultados de la auditoría inicial, puede construirse una tabla resumen:

```
1 resumen_descarga = pd.DataFrame({
2     "Elemento": [
3         "Ticker",
4         "Fecha inicial solicitada",
5         "Fecha final exclusiva solicitada",
6         "Primera fecha disponible",
7         "Ultima fecha disponible",
```

```

8     "Numero de observaciones",
9     "Numero de columnas",
10    "Total de valores faltantes",
11    "Fechas duplicadas",
12    "Fechas ordenadas"
13 ],
14 "Resultado": [
15     ticker,
16     fecha_inicio,
17     fecha_fin,
18     datos.index.min().strftime("%Y-%m-%d"),
19     datos.index.max().strftime("%Y-%m-%d"),
20     len(datos),
21     datos.shape[1],
22     datos.isna().sum().sum(),
23     datos.index.duplicated().sum(),
24     datos.index.is_monotonic_increasing
25 ]
26 })
27
28 resumen_descarga

```

Esta tabla puede incorporarse posteriormente al reporte parcial del proyecto.

16. Conservación de los datos originales

Una práctica fundamental en análisis de datos consiste en conservar una copia sin modificaciones de la descarga original. Se recomienda utilizar la estructura:

Carpeta	Contenido
data/raw	Datos originales descargados.
data/processed	Datos limpiados o transformados.
notebooks	Cuadernos de Jupyter.
figures	Gráficas exportadas.

La carpeta para datos originales puede crearse con:

```

1 ruta_raw = Path("data/raw")
2 ruta_raw.mkdir(parents=True, exist_ok=True)

```

Después se guarda la tabla:

```

1 archivo_salida = ruta_raw / "SPY_2015_2025_raw.csv"
2
3 datos.to_csv(archivo_salida)
4
5 print("Archivo guardado en:", archivo_salida)

```

El sufijo **raw** indica que se trata de datos sin limpiar.

Recuperación del archivo

Guardar un archivo no es suficiente: también conviene comprobar que puede leerse correctamente.

```
1 datos_recuperados = pd.read_csv(  
2     archivo_salida,  
3     index_col=0,  
4     parse_dates=True  
5 )  
6  
7 datos_recuperados.head()
```

Se comparan las dimensiones:

```
1 print("Dimensiones originales:", datos.shape)  
2 print("Dimensiones recuperadas:", datos_recuperados.shape)
```

Finalmente:

```
1 assert datos.shape == datos_recuperados.shape  
2  
3 print("El archivo se guardo y recupero correctamente.")
```

La instrucción `assert` genera un error si la condición es falsa. En este caso permite comprobar que el número de filas y columnas no cambió durante el proceso de guardado y lectura.

Regla del proyecto. El archivo `SPY_2015_2025_raw.csv` no debe sobrescribirse con datos limpiados. La base procesada de la Semana 4 deberá guardarse con un nombre distinto.

17. Ficha técnica de la descarga

Al finalizar la exploración debe completarse la siguiente ficha:

Elemento	Resultado
Activo	SPY.
Fuente	Yahoo Finance.
Herramienta	yfinance.
Frecuencia solicitada	Diaria.
Fecha inicial solicitada	2015-01-01.
Fecha final exclusiva solicitada	2026-01-01.
Primera fecha disponible	Completar con Python.
Última fecha disponible	Completar con Python.
Número de observaciones	Completar con Python.
Número de columnas	Completar con Python.
Valores faltantes	Completar con Python.
Fechas duplicadas	Completar con Python.
Fechas ordenadas	Completar con Python.
Observaciones en fin de semana	Completar con Python.
Fecha de descarga	Completar al ejecutar.
Versión de yfinance	Completar al ejecutar.
Nombre del archivo original	SPY_2015_2025_raw.csv.

Esta ficha documenta no solo el contenido de la base, sino también las condiciones bajo las cuales fue obtenida.

18. Producto esperado al final de la semana

La estudiante deberá entregar:

1. el notebook 03_descarga_exploracion_inicial.ipynb;
2. el archivo data/raw/SPY_2015_2025_raw.csv;
3. una gráfica del precio de cierre ajustado;
4. una gráfica del volumen diario;
5. la tabla de valores faltantes;
6. la tabla resumen_descarga;
7. la ficha técnica de la descarga;
8. una conclusión breve, de aproximadamente media cuartilla, sobre la cobertura y calidad inicial de la base.

19. Preguntas de repaso

1. ¿Qué representa cada fila de la base descargada?

2. ¿Qué representa cada columna?
3. ¿Qué es el índice de un `DataFrame`?
4. ¿Cuál es la diferencia entre `Close` y `Adj Close`?
5. ¿Por qué se escribe 2026-01-01 como fecha final de la descarga?
6. ¿Qué función cumple el argumento `interval="1d"`?
7. ¿Por qué se especifica `auto_adjust=False`?
8. ¿Qué significa que una tabla esté vacía?
9. ¿Qué información devuelve `datos.shape`?
10. ¿Qué información proporciona `datos.info()`?
11. ¿Qué es un `DatetimeIndex`?
12. ¿Qué significa que el índice sea monótonamente creciente?
13. ¿Por qué no hay una observación para cada día natural?
14. ¿Cuál es la diferencia entre una fecha ausente y un valor `NaN`?
15. ¿Cómo se detectan fechas duplicadas?
16. ¿Qué relaciones deberían satisfacer `Open`, `High`, `Low` y `Close`?
17. ¿Por qué no se debe eliminar automáticamente una observación con volumen extremo?
18. ¿Por qué se conserva una copia de los datos originales?
19. ¿Qué diferencia existe entre `data/raw` y `data/processed`?
20. ¿Qué problemas se dejarán para la Semana 4?

20. Conclusión

En esta semana se construyó formalmente la primera base de datos del proyecto. La descarga de `SPY` se realizó mediante `yfinance`, utilizando un periodo diario de 2015 a 2025 y conservando separadas las columnas de cierre y cierre ajustado.

La exploración inicial se concentró en la estructura del `DataFrame`, la interpretación de sus columnas, la cobertura temporal, las diferencias entre fechas consecutivas, los valores faltantes, las fechas duplicadas y algunas condiciones elementales de consistencia. También se propusieron gráficas preliminares del precio ajustado y del volumen.

El resultado principal de la semana no es una base limpia, sino una base **documentada**. Antes de transformar los datos, se debe saber qué información contienen y qué problemas podrían requerir atención. La copia original se conservará en la carpeta `data/raw`, mientras que cualquier versión modificada se almacenará más adelante en `data/processed`.

La Semana 4 se dedicará a la limpieza formal de la base: ordenamiento temporal, revisión detallada de fechas, tratamiento de valores faltantes, resolución de duplicados y construcción de un archivo procesado reproducible.

Bibliografía mínima sugerida

- McKinney, W. *Python for Data Analysis*. O'Reilly Media.
- Tsay, R. S. *Analysis of Financial Time Series*. Wiley.
- Brooks, C. *Introductory Econometrics for Finance*. Cambridge University Press.
- pandas. Documentación oficial: <https://pandas.pydata.org/docs/>.
- yfinance. Documentación oficial de yf.download.
- Matplotlib. Documentación oficial: <https://matplotlib.org/stable/>.